

Advanced Machine Learning: Bayes, Risk, Overfitting, and Regression

Document Summary

This note covers foundational and advanced concepts in machine learning, starting with Bayes' Rule and the Naïve Bayes Classifier, then exploring evaluation metrics like true risk and Bayes risk, and addressing overfitting through cross-validation and regularization. It delves into the PAC learning framework, model selection techniques, and practical applications in linear regression, including least squares optimization and extensions to logistic regression for classification. Key examples and visualizations illustrate probabilistic reasoning, generalization challenges, and optimization methods for robust ML models.

Advanced Machine Learning: Bayes, Risk, Overfitting, and Regression

Bayes' Rule and Classifiers

One Step Backward: Bayes' Rule

Bayes' Rule

This section reviews **Bayes' Rule** as a foundational concept, drawing from Slide 1 of A.A. 2025/2026 Advanced Machine Learning (AM ML 1). The reference material is available at <http://www-inst.eecs.berkeley.edu/~cs188/fa19/>. Key elements include the proportionality symbol $\propto$, the negation or error indicator $\times$, and core terms like **posterior**, **prior**, and **likelihood**. These components form the basis for probabilistic reasoning in **Machine Learning**.

To clarify **Bayes' Rule**, recall that it updates probabilities based on new evidence. Mathematically, the posterior is proportional to the likelihood times the prior:

Bayes' Rule Equation

$$P(\theta|D) \propto P(D|\theta) \cdot P(\theta)$$

Bayes' Rule Application

Suppose you have a prior belief that a coin is fair ($P(\theta = 0.5) = 0.8$). Observing 3 heads in 5 flips (likelihood $P(D|\theta)$) updates this to a posterior via **Bayes' Rule**, yielding a refined estimate closer to $\theta = 0.6$.

Slide credits go to Barnabás Póczos & Alex Smola for related concepts in the Naïve Bayes Classifier.

One Step Backward: Naïve Bayes Classifier

Naïve Bayes Classifier

Building on **Bayes' Rule**, this introduces the **Naïve Bayes Classifier**, a simple yet effective probabilistic model that assumes feature independence. It appears in Slide 4, with credits to Barnabás Póczos & Alex Smola. The classifier computes the posterior probability of a class given features by multiplying priors and likelihoods under the naïve independence assumption.

For instance, in spam detection, features like word counts are treated as independent, allowing quick computation:

Naïve Bayes Posterior

$$P(\text{Spam}|\text{words}) = \frac{P(\text{words}|\text{Spam})P(\text{Spam})}{P(\text{words})}$$

This simplifies complex joint distributions, making it computationally efficient. #computer-science/machine-learning #level/advanced

One Step Forward: Deep Learning

Transitioning from classical probabilistic models, this section moves toward **Deep Learning**, highlighting how **Neural Networks** extend these ideas to hierarchical feature learning. It is covered in Slide 5, serving as a bridge to more advanced architectures. See **Deep Learning** for further details.

Evaluating Predictions: The Avocado Problem

How Good Are Our Predictions?

Prediction Evaluation

A central question in machine learning is: Why focus on minimizing `training error` when our ultimate goal is low `test error` on unseen data? Achieving 100% accuracy is not always feasible due to inherent uncertainties in real-world data. This section explores key parameters that define a learning problem, such as data distribution and noise levels. Slide credits: Francesco Orabona, with imagery from <https://www.istockphoto.com/>.

To evaluate predictions effectively, we must distinguish between performance on known data (training) and generalization to new data (testing). Over-optimism on training alone can mislead model quality.

The Avocado Problem: Example

Avocado Ripeness Prediction

Consider a practical scenario: buying your first avocado and needing to predict ripeness without cutting it open. How do you determine if it is ripe? To build intuition, gather data by buying several avocados and inspecting them internally. Relevant features include:

- `color` : Ranging from dark green (unripe) to dark brown (overripe).
- `softness` : Varying from rock hard to mushy.

For a new avocado, use these features to predict ripeness. Is perfect prediction possible? No, due to natural variations. The process involves: `Training Data` → Build a `Model` → Make predictions on a new sample. Slides 6–8 emphasize this example and pose key questions about feasibility.

Suppose training data shows green, hard avocados are unripe (80% cases) and brown, soft ones are ripe (90% cases). A new green-soft avocado might be predicted as unripe, but with some error risk.

Feature	Unripe Example	Ripe Example	Overripe Example
Color	Dark Green	Light Brown	Dark Brown
Softness	Rock Hard	Soft	Mushy

This table illustrates feature variations, highlighting why models must handle overlaps for robust predictions. `#note/example`

Notation and Fundamental Concepts

Notation

This section establishes basic notation for **Machine Learning**, essential for consistent communication. It is introduced in Slide 9, credit: Francesco Orabona. Common symbols include x for inputs, y for labels, and $h(x)$ for predictions. Prerequisite: **Linear Algebra**.

Loss

Loss

The **loss** quantifies prediction errors, guiding model optimization. For a prediction $\hat{y}$ and true y , loss measures discrepancy, e.g., squared error $(y - \hat{y})^2$. Its role is to penalize poor fits, as detailed in Slide 10, credit: Francesco Orabona.

True Risk

True Risk

The **True Risk** is the expected loss over the true underlying data distribution:

True Risk Equation

$$R(h) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[L(h(x), y)]$$

where $\mathcal{D}$ is the true distribution. This represents average performance on infinite unseen data, covered in Slides 11 and 12, credit: Francesco Orabona.

Bayes Classifier

Bayes Classifier

The optimal classifier minimizes true risk by choosing the class with highest posterior probability:

Bayes Classifier

$$h^*(x) = \arg \max_y P(y|x)$$

This **Bayes Classifier** achieves the lowest possible error under the true distribution, as explained in Slide 13, credit: Francesco Orabona.

Bayes Risk

Bayes Risk

The **Bayes Risk** is the minimum achievable true risk using the **Bayes Classifier**:

Bayes Risk

$$R^* = \min_h R(h)$$

It sets a theoretical lower bound, often unattainable in practice due to unknown distributions. See Slide 14, credit: Francesco Orabona.

Bayes Risk Illustration

In binary classification with equal priors, if $P(y = 1|x) = 0.7$, the **Bayes Classifier** predicts 1, and **Bayes Risk** reflects irreducible error from noisy labels.

Learning Framework

Batch Learning

Batch Learning

Batch Learning involves training on a fixed dataset all at once, without online updates. The process: Collect data, select a hypothesis class, minimize empirical risk. Slide 15, credit: Francesco Orabona.

Batch Learning - IID Condition

A key assumption is that data samples are Independent and Identically Distributed (IID) from the true distribution. This ensures training data represents the test distribution reliably. Slide 16, credit: Francesco Orabona.

Empirical Risk

Empirical Risk

The **Empirical Risk** approximates true risk as the average loss on finite training data:

Empirical Risk Equation

$$\hat{R}(h) = \frac{1}{n} \sum_{i=1}^n L(h(x_i), y_i)$$

It is computable and used for optimization, per Slide 17, credit: Francesco Orabona.

Can Only Be Probably Correct

Learning guarantees are inherently probabilistic because data is random. We cannot guarantee zero error with certainty, only high probability. Slide 18, credit: Francesco Orabona.

Can Only Be Approximately Correct

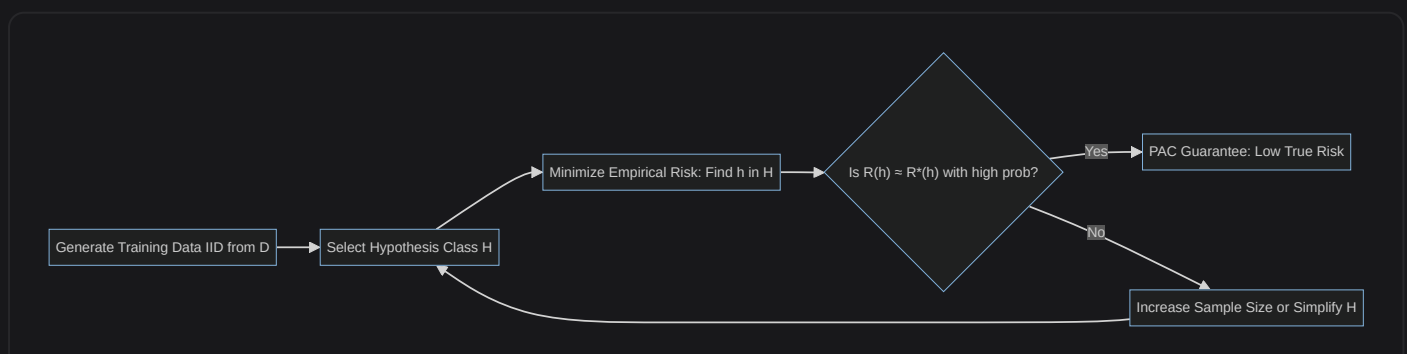
Empirical risk provides an approximation to true risk, with bounds on deviation based on sample size. Slide 19, credit: Francesco Orabona.

Probably Approximately Correct (PAC) Learning

PAC Learning

The **PAC Learning** framework formalizes this: With probability at least $1 - \delta$ over data samples, the empirical risk minimizer $\hat{h}$ satisfies $R(\hat{h}) \leq R(h) + \epsilon$ for small $\epsilon > 0$. This ensures the learned model is close to optimal with high confidence. Slides 20 and 21, credit: Francesco Orabona. References: <http://www-inst.eecs.berkeley.edu/~cs188/fa19/>.

To visualize the **PAC Learning** process:



This diagram shows the iterative nature of achieving PAC bounds. #note/process

Overfitting and Underfitting

Things Can Go Wrong

Underfitting and Overfitting

In practice, models can fail by underfitting (too simple, missing patterns) or overfitting (too complex, capturing noise). Examples from `Machine Learning and Pattern Recognition` by Bishop illustrate **regression** with `polynomial curves`: Low-degree polynomials underfit by ignoring trends, while high-degree ones overfit by fitting noise. This is the general `overfitting` phenomenon. Slides 22–27, credits: Bishop, Goodfellow et al. (`Deep Learning` book), <http://www-inst.eecs.berkeley.edu/~cs188/fa19/>. Figure credits: Nati Srebro.

Polynomial Fitting Example

Fitting a line (degree 1) to quadratic data underfits; a degree 10 polynomial wiggles through every point but fails on new data.

Overfitting

Overfitting

Overfitting occurs when a model performs well on training data but poorly on test data, memorizing specifics rather than generalizing. It arises from excessive model complexity relative to data size. Detailed in Slide 28, credit: Francesco Orabona. See `Cross-Validation` for mitigation.

Model Selection: Cross-Validation

Cross Validation

Cross Validation

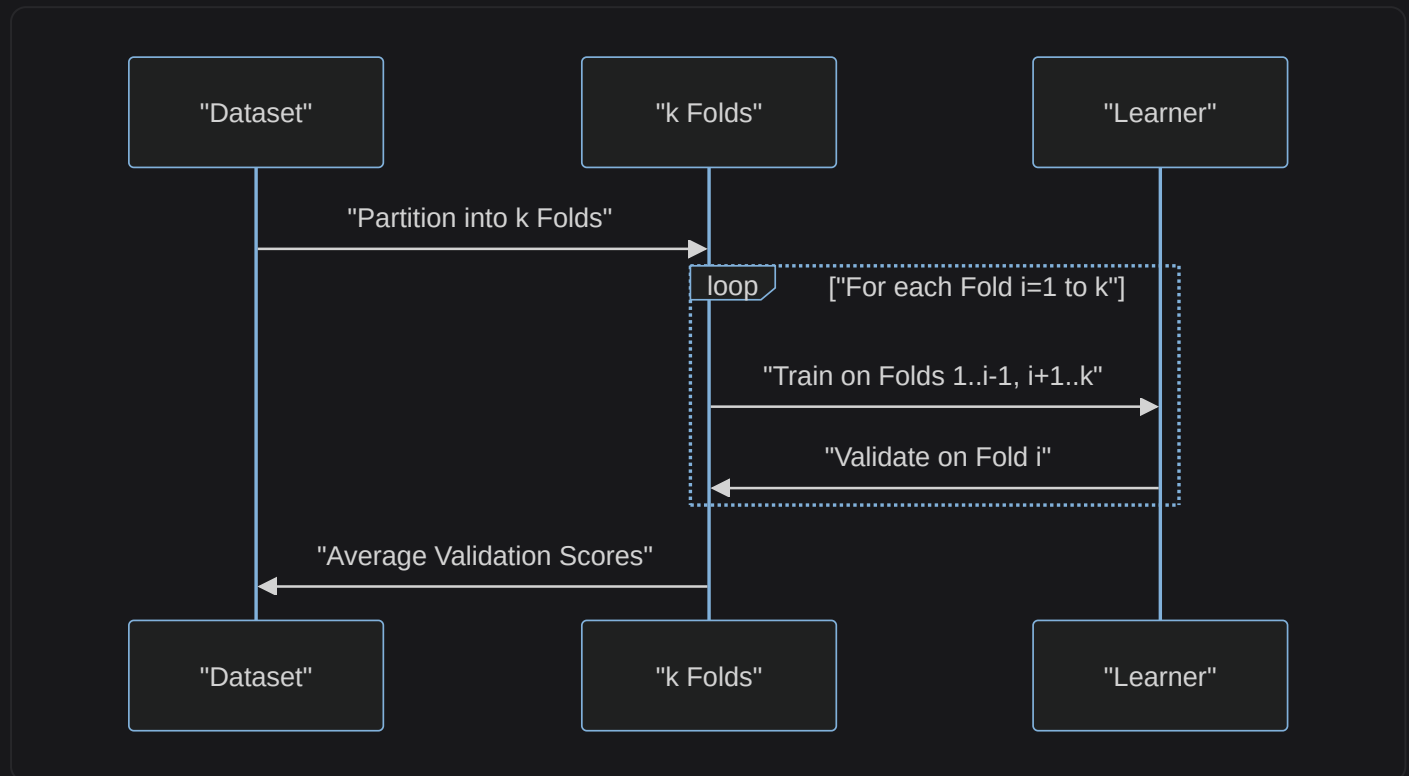
To select models reliably, split the dataset into `Training`, `Testing`, and `Validation` sets. When data is limited, use **Cross Validation** to reuse samples efficiently. This involves training multiple times on different subsets. Figure from Bishop's book. Slides 29–32.

`#note/process`

k-fold Cross Validation

In k-fold Cross Validation, partition data into k equal folds. For each iteration, train on k-1 folds and validate on the held-out fold; average results. If k equals the sample size |S|, it becomes **Leave-One-Out Cross Validation**, exhaustive but computationally intensive. Standard splits include **Training** for fitting, **Cross Validation** for tuning, and **Testing** for final evaluation. Figures from Bishop. Slides 30–32.

The process can be visualized as:



This sequence diagram clarifies the rotational validation steps.

Train - Validation - Test

The standard split allocates data as: Training (60-80%) for learning parameters, Validation (10-20%) for hyperparameter tuning, and Test (10-20%) for unbiased evaluation. This prevents overfitting to the test set. Slide 33.

Model Selection In Summary

Model selection techniques include hold-out validation, k-fold cross-validation, and leave-one-out, chosen based on data size and computational budget. They balance bias and variance for robust generalization. Slides 34–35, reference: <https://en.ephoto360.com/blood-writing-text-online-77.html>.

Solutions to Overfitting: Regularization

Learning Solution to Overfitting

To combat overfitting, control model complexity through **regularization**, which penalizes overly intricate hypotheses. This encourages simpler models that generalize better. Figure credit: Nati Srebro. Slide 36.

Regularization

The regularized objective minimizes empirical loss plus a complexity penalty:

Regularized Objective

$$\hat{h} = \arg \min_h \hat{R}(h) + \lambda \cdot \text{Complexity}(h)$$

Here, $\lambda > 0$ balances fit and simplicity; the form of `complexity / regularization` varies by hypothesis space (e.g., parameter norms). Still, select λ via `cross validation` for best validation performance. Slides 37–38.

Regularization Effect

For $\lambda = 0$, no penalty leads to overfitting; $\lambda = 10$ smooths the model, reducing variance.

Linear Regression

Linear Regression

Linear Regression

Linear Regression models the relationship between inputs and outputs as a linear function, predicting continuous values. It is a cornerstone of supervised learning. Slides 39–40, credit: Francesco Orabona. See [Supervised Learning](#).

Flashback: Loss Function

Revisiting loss in regression: Typically, squared error measures deviation from true values, promoting smooth fits. Slide 41, credit: Francesco Orabona.

Empirical Loss

Compute empirical loss as the mean squared error over training data:

Empirical Loss in Regression

$$\hat{R}(\theta) = \frac{1}{n} \|y - X\theta\|^2$$

Slide 42, credit: Francesco Orabona.

Linear Predictors in 1D

Start with simple 1D linear models: $h(x) = \theta x$, ignoring bias initially for clarity, then incorporate it as θ_0 . This builds intuition before multivariate extensions. Slides 43–54, credits: Francesco Orabona, William Cohen, E. Rodolà.

Linear Fitting to Data

The fitting process involves selecting θ to minimize errors, visualized with scatter plots and fitted lines. Examples show how lines adapt to data trends. Slides 44–47, credit: Francesco Orabona.

Linear Functions

The general form is $h(x) = \theta^T x$, linear in parameters θ . Slide 48, credit: Francesco Orabona.

Least Squares Criterion

Optimize by minimizing the sum of squared residuals:

Least Squares Criterion

$$\min_{\theta} \sum_i (y_i - \theta^T x_i)^2$$

This yields the best linear unbiased estimator under Gaussian noise. Slide 49, credit: Francesco Orabona.

Least Squares in Matrix/Vector Form

The closed-form solution is:

Least Squares Solution

$$\hat{\theta} = (X^T X)^{-1} X^T y$$

assuming $X^T X$ is invertible. Slide 50, credit: Francesco Orabona.

Here's a Python implementation for clarity:

```
import numpy as np

# Example data: X (features with bias column), y (targets)
X = np.array([[1, 1], [1, 2], [1, 3]]) # 1 is bias term
y = np.array([1, 2, 3])

# Least squares solution
theta_hat = np.linalg.inv(X.T @ X) @ X.T @ y
print("Estimated theta:", theta_hat) # Outputs approx [0, 1]
```

This code computes $\hat{\theta}$ for a simple line $y = x$.

Least Squares via Calculus

Derive the solution by setting the gradient to zero:

Gradient Derivation

$$\nabla_{\theta} \|y - X\theta\|^2 = -2X^T(y - X\theta) = 0$$

Solving gives the matrix form. Slides 51–52, credits: E. Rodolà, Francesco Orabona.

Add Regularization (Ridge Regression)

Incorporate L2 regularization:

Ridge Regression

$$\min_{\theta} \|y - X\theta\|^2 + \lambda \|\theta\|^2$$

Solution: $\hat{\theta} = (X^T X + \lambda I)^{-1} X^T y$. This shrinks parameters, reducing overfitting. Slide 55.

Beyond Linear Models: Polynomial Regression

Extend to nonlinear forms using basis functions, e.g., $\phi(x) = [1, x, x^2]$ for quadratics. The model remains linear in θ : $h(x) = \theta^T \phi(x)$. Slides 56–57, credit: Francesco Orabona.

Polynomial Features

For $x = [1, 2, 3]$, $\phi(x) = [1, x, x^2]$ yields features $[1, 1, 1]$, $[1, 2, 4]$, $[1, 3, 9]$; fit linear model on these.

Summary of Linear Regression

In summary, **PAC** learning contrasts true vs. empirical risk, using **loss** and **empirical risk minimization** (ERM) with probabilistic guarantees. Address underfitting/overfitting via **cross-validation**. **Linear Regression** applies ERM with square loss, solved efficiently by matrix inversion. Basis functions (e.g., polynomials) enable nonlinearity while staying linear in parameters. Include **regularization** as needed. Slide 58. [#note/summary](#)

Using Regression for Classification

Use a Regression Model for Classification

Adapt linear regression for binary classification by thresholding outputs: Predict class 1 if $h(x) > \Theta$, else 0. This leverages regression's linearity for decision boundaries. Slide 59, credit: E. Rodolà.

Logistic Regression

Logistic Regression

For probabilistic classification, apply the sigmoid:

Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where $z = \theta^T x$. The loss is **cross-entropy** (log loss):

Cross-Entropy Loss

$$L(\theta) = -\frac{1}{n} \sum [y_i \log \sigma(\theta^T x_i) + (1 - y_i) \log(1 - \sigma(\theta^T x_i))]$$

Slides 60–66, 73, credits: E. Rodolà. Compared to SVM's hinge loss:

- Hinge: $\max(0, 1 - y\theta^T x)$, margin-focused.

References: <https://towardsdatascience.com/optimization-loss-function-under-the-hood-part-iii-5dff33fa015d>. Slides 68–71. See [Support Vector Machines](#).

Sigmoid Output

For $z = 2$, $\sigma(2) \approx 0.88$, indicating 88% probability of class 1.

Logistic Regression: Finding a Solution

No closed form; use optimization like gradient descent:

Gradient Descent Update

$$\theta \leftarrow \theta - \eta \nabla L(\theta)$$

where $\nabla L(\theta) = \frac{1}{n} X^T (\sigma(X\theta) - y)$. Iterate until convergence. Detailed steps include initialization, updates, and convergence checks, with examples. Slides 74–93, credit: E. Rodolà. Empty slides (67, 72) mark transitions.

For implementation:

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Gradient descent for logistic regression
def logistic_gradient_descent(X, y, eta=0.01, epochs=1000):
    theta = np.zeros(X.shape[1])
    for _ in range(epochs):
        z = X @ theta
        grad = (1 / len(y)) * X.T @ (sigmoid(z) - y)
        theta -= eta * grad
    return theta

# Usage example
X = np.array([[1, 1], [1, 2]]) # With bias
y = np.array([0, 1])
```

```
theta = logistic_gradient_descent(X, y)
print("Learned theta:", theta)
```

This code performs basic gradient descent, converging to separating parameters.

References

- [Machine Learning](#)
- [Deep Learning](#)
- [Linear Algebra](#)
- [Supervised Learning](#)
- [Support Vector Machines](#)